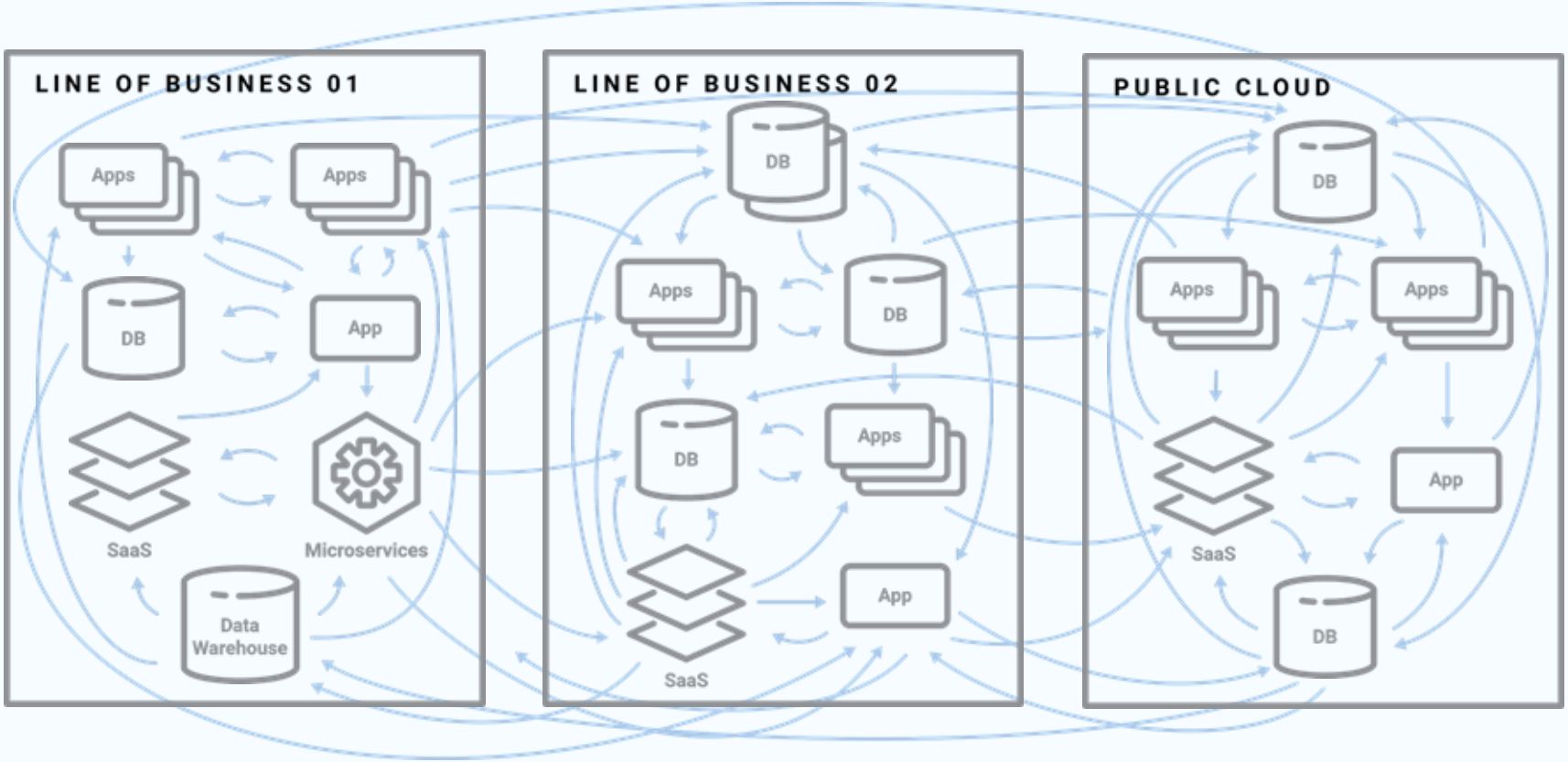


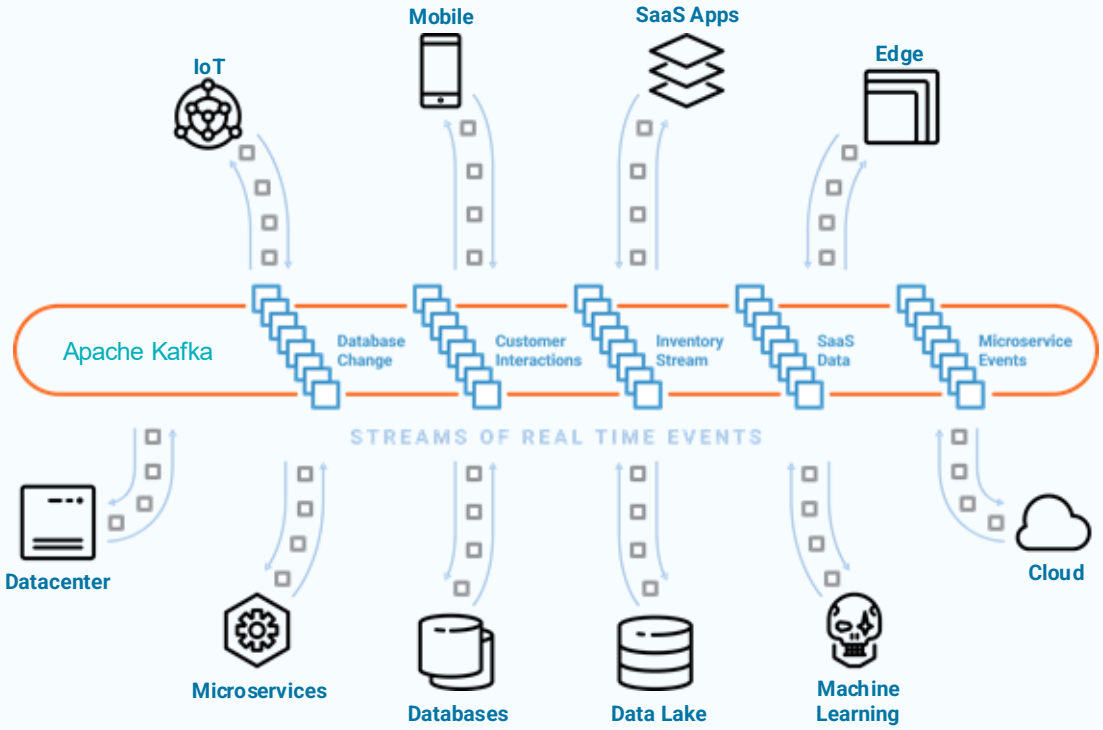
Mani Selvan K
Solutions Engineering

Pre-Streaming



Apache Kafka®: the De-facto Standard for Real-Time Event Streaming

- Events-first design
- Break Silos
- Write once, consume many
- Very large scale
- Strong decoupling



Introduction to Streaming

Customers demand immediacy in every aspect of their lives through real-time applications

Transportation



Connect to a driver immediately after rideshare request

Retail



Instant notification upon package delivery

Technology



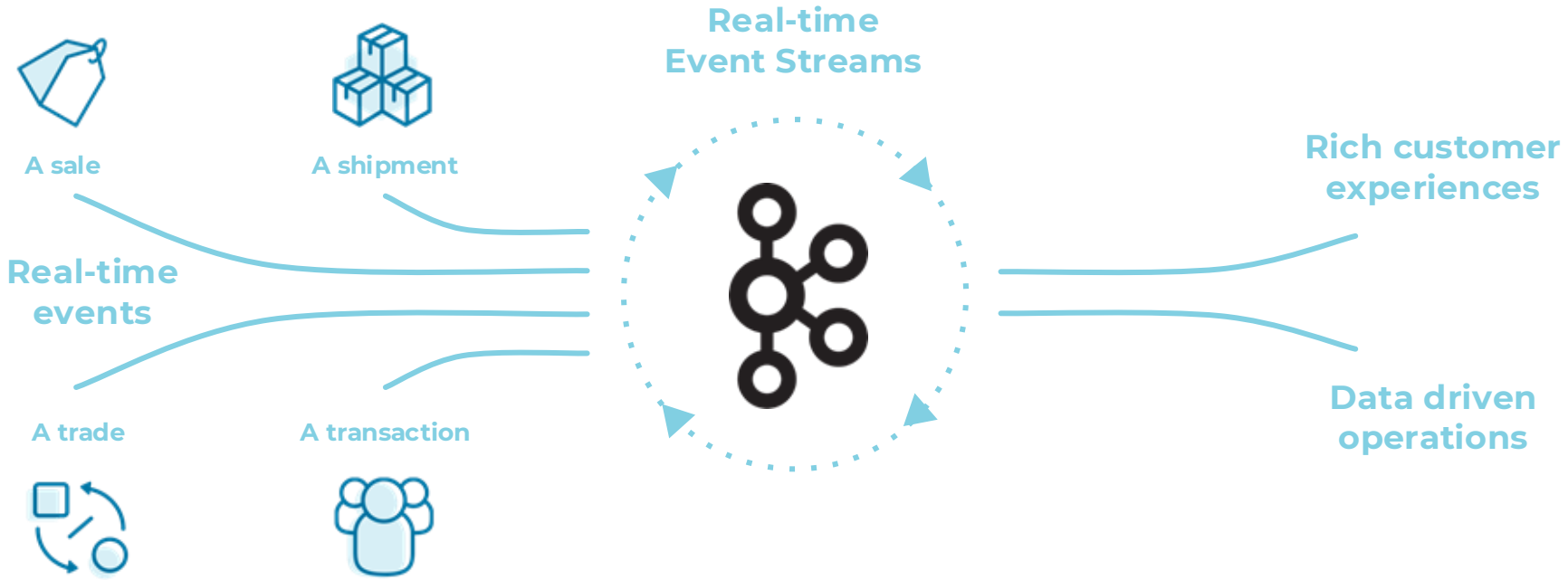
Real-time notification once a new patch is available

Banking



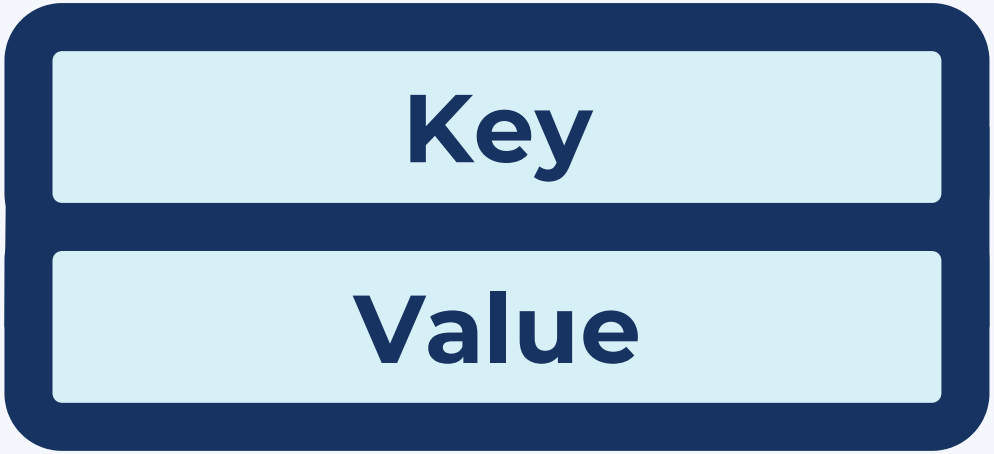
Automatic alert once fraudulent activity has been detected

This new paradigm shift, for data in motion, is centered around event streams



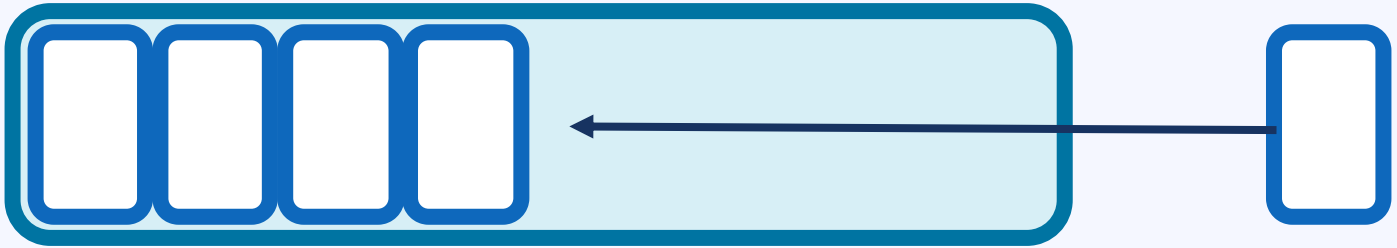
So what is Apache Kafka®?

Message / Event

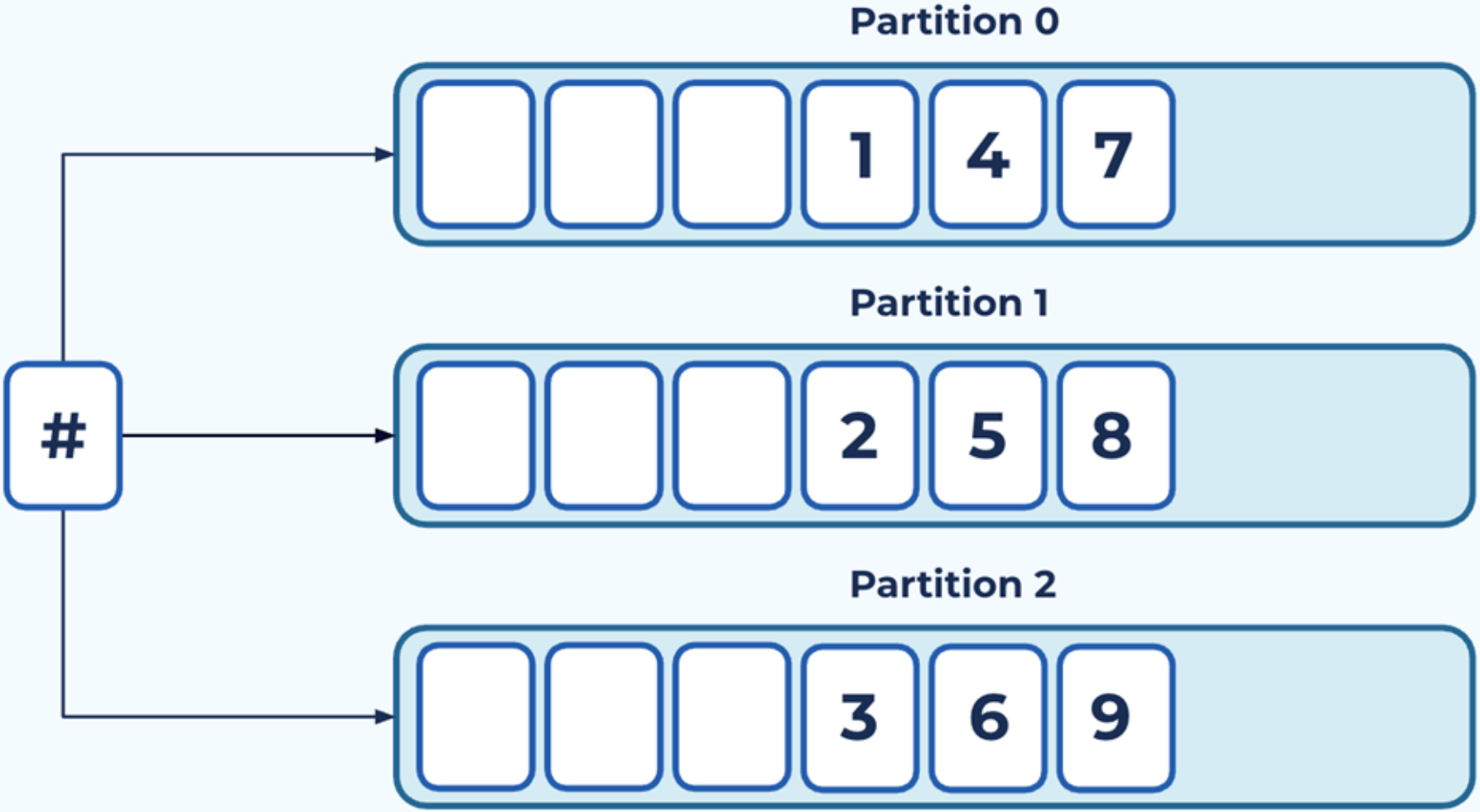


Topics

Topics



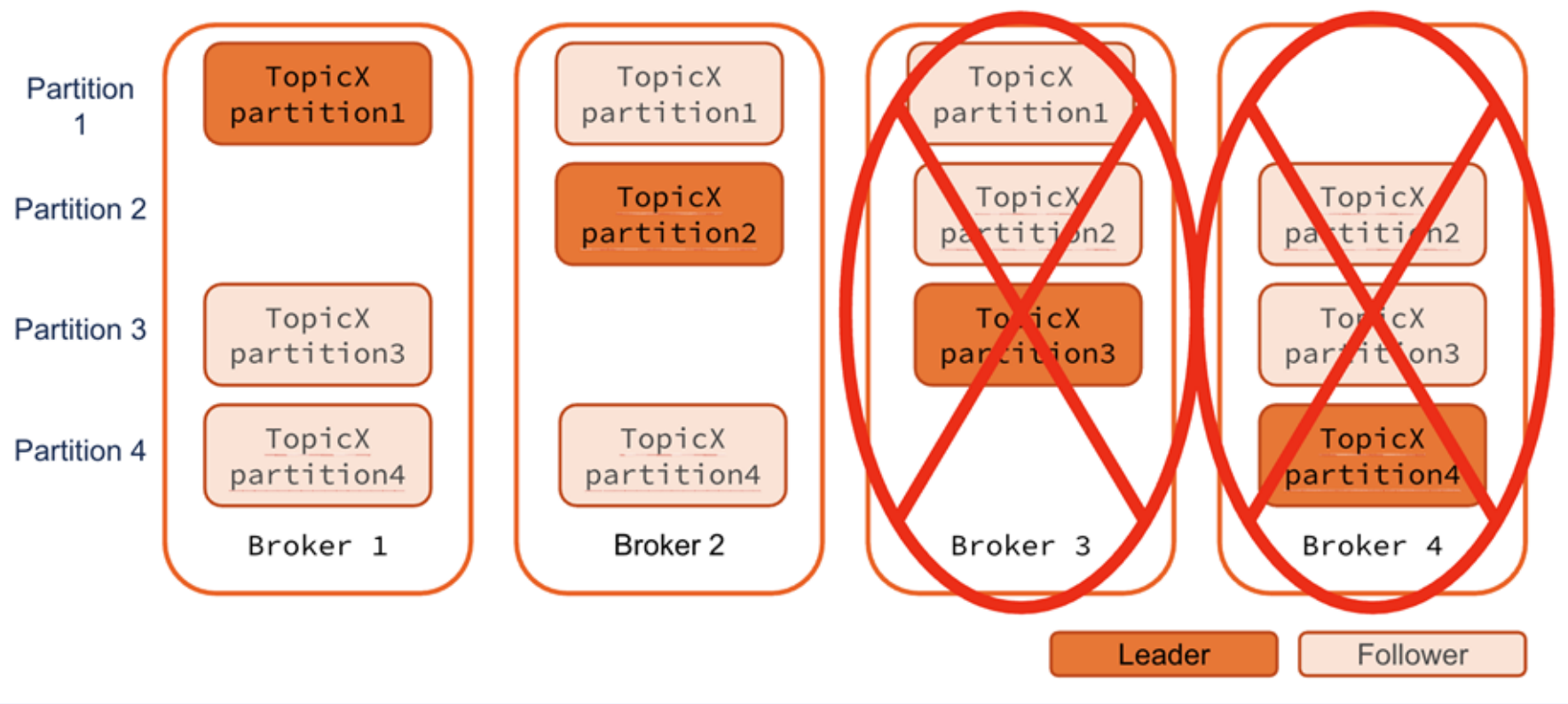
Partitioning


Partition 0
1
4
7
Partition 1
#
2
5
8
Partition 2
3
6
9

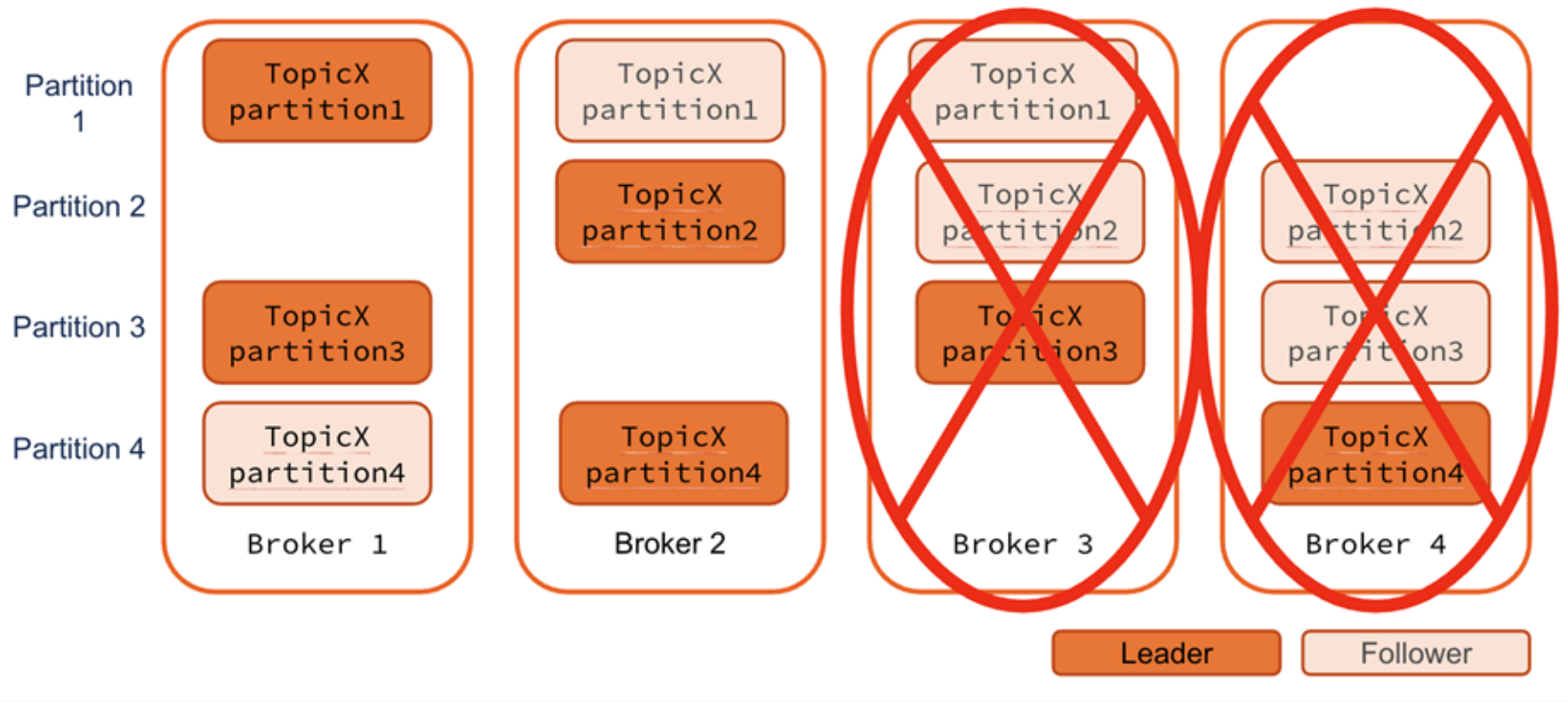
Brokers

& Replication

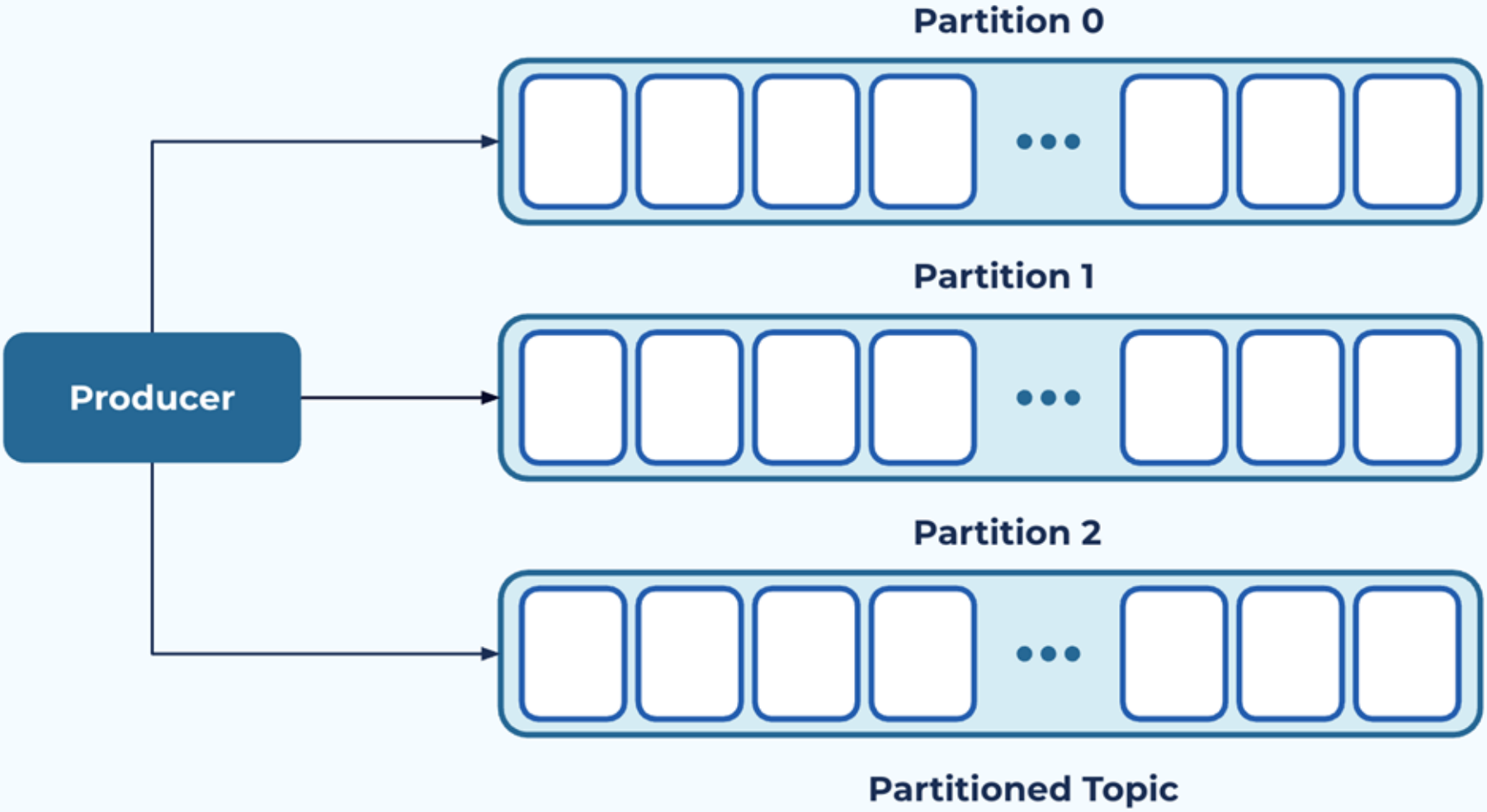
Partition Leadership and Replication



Partition Leadership and Replication



Producers


Partition 0
Partition 1
Partition 2
Producer
Partitioned Topic

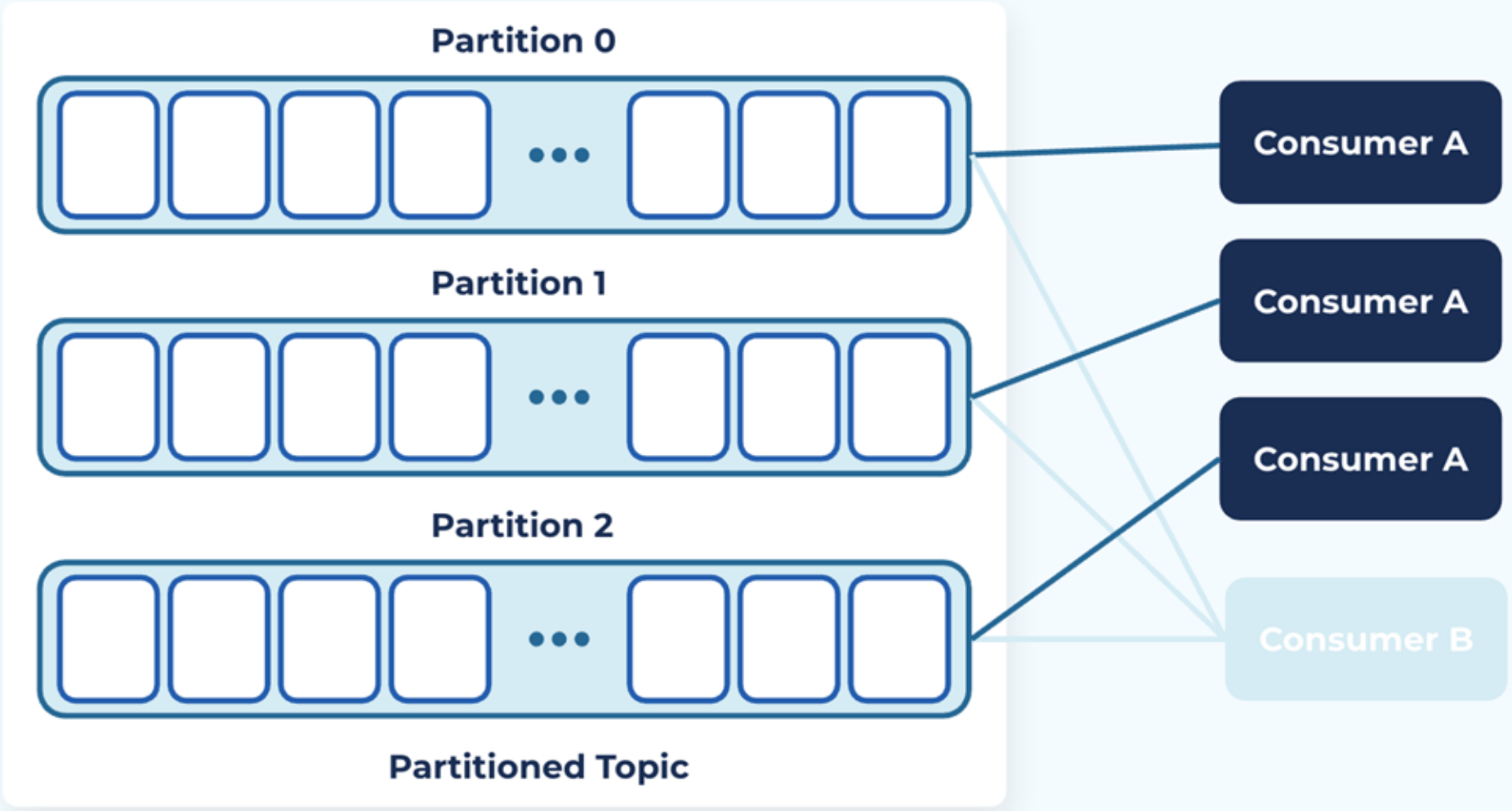
Delivery Semantics:

- Kafka guarantees at-least-once delivery by Default
- Exactly-Once delivery can be configured using Transactions

Message Ordering:

- Messages are split into Partitions using Keys;
- Messages with the same Key are always written into the Same Partition in the order they were received(and an offset is attached to the message)
- Consumers reading form the same Partition will always get messages in Order

Consumers


Partition 0
Partition 1
Partition 2
Partitioned Topic
Consumer A
Consumer A
Consumer A
Consumer B

Behavior:

- Pull-based applications reading from a Topic
- Once a message is successfully read, commits the offset of that message to the offset topic
- Multiple Consumers can read from the same Topic
- Multiple instances of the same consumer client can be deployed as a *group* for parallel consumption
- Consumers manage their own offsets and can rewind or jump to arbitrary points in the log. The broker doesn't enforce tracking.
- Event Replay can be triggered by a consumer at any point in time



CONFLUENT
Developer

Apache Flink® 101

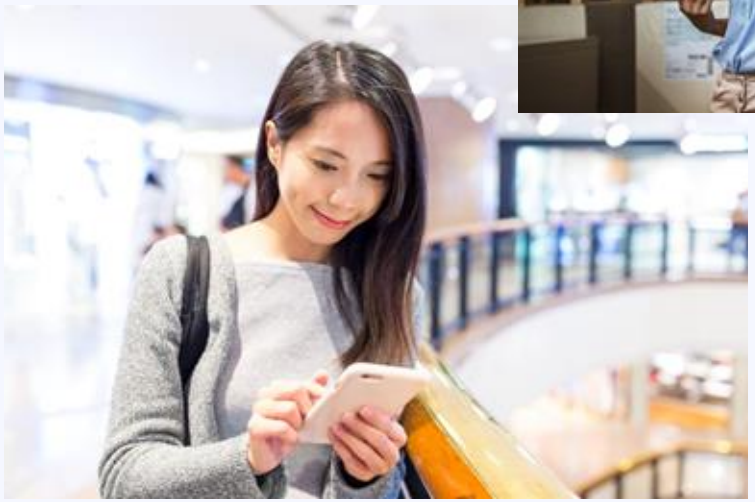
CONFLUENT Developer

Agenda

01 Intro to Stream Processing with Apache Flink
02 Intro to Flink SQL
03 The Flink Runtime
04 Using Kafka with Flink
05 Stateful Stream Processing with Flink SQL
06 Event Time and Watermarks
07 Checkpoints and Recovery
08 Q & A

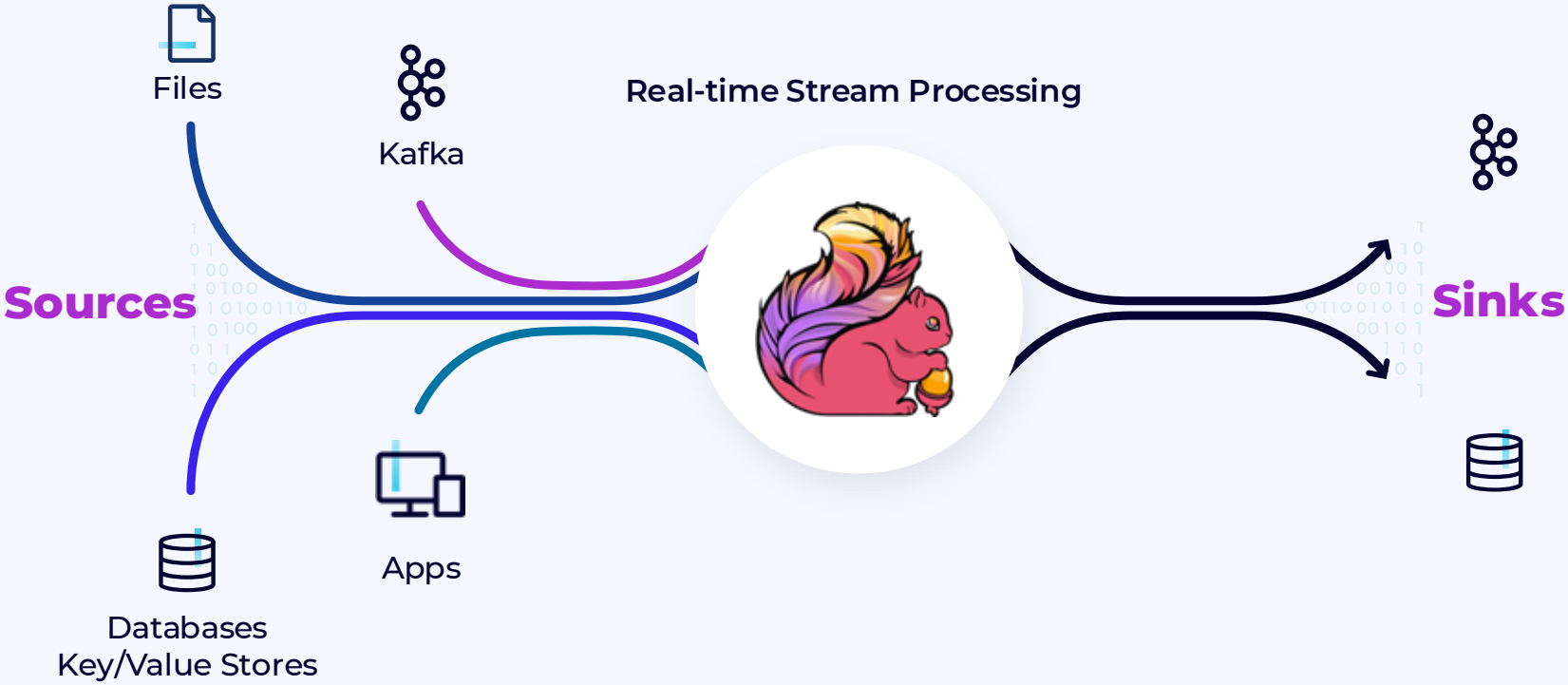
Today’s consumers expect real-time services







Real-time services rely on stream processing



Developers are turning to Flink because of its performance and rich feature set



Elastic Scalability

Flink is capable of supporting stream processing workloads at tremendous scale



Language Flexibility

Flink supports Java, Python, & SQL, enabling developers to work in their language of choice



Unified Processing

Flink supports stream processing, batch processing, and ad-hoc analytics through one technology

Flink is a top 5 Apache project and has a very active community

The four cornerstones on which Flink is built



Streaming



State



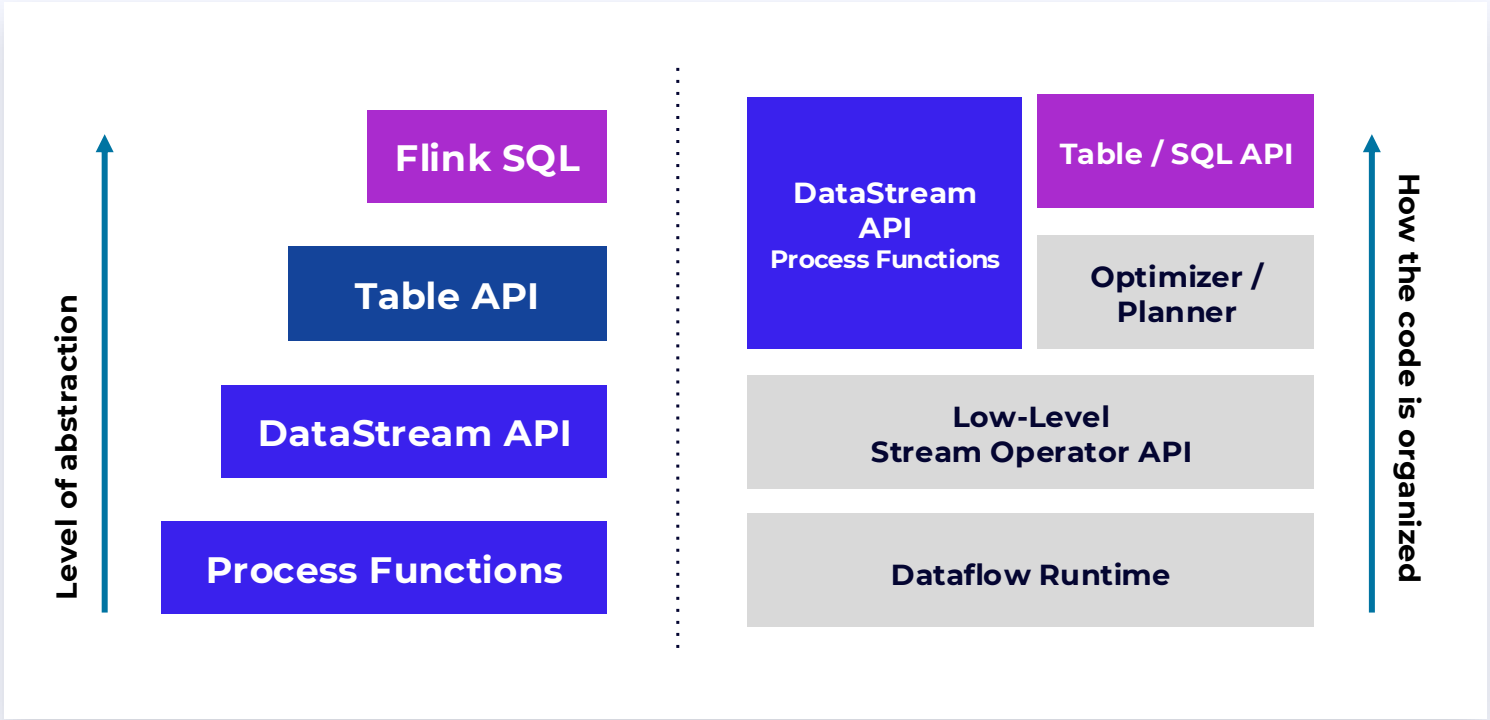
Time



Snapshots

Flink's APIs

CONFLUENT Developer



Example

CONFLUENT Developer

```
CREATE TABLE Shipments
  (item STRING, count INT)
  WITH (...)
```

item	count
HAT	50
MUG	10
HAT	-15

Example

CONFLUENT Developer

```
CREATE TABLE Shipments
  (item STRING, count INT)
  WITH (...)
```

item	count
HAT	50
MUG	10
HAT	-15

```
CREATE TABLE Inventory
  (item STRING, stock INT)
  WITH (...)
```

item	stock
HAT	35
MUG	10

Example

CONFLUENT Developer

```
CREATE TABLE Shipments
  (item STRING, count INT)
  WITH (...)
```

```
INSERT INTO Inventory
SELECT item, SUM(count) AS stock
FROM Shipments
GROUP BY item
```

```
CREATE TABLE Inventory
  (item STRING, stock INT)
  WITH (...)
```

item	count
HAT	50
MUG	10
HAT	-15

item	stock
HAT	35
MUG	10

Example

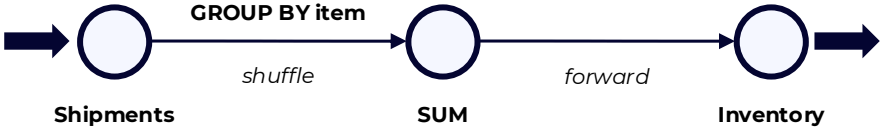
CONFLUENT Developer

```
CREATE TABLE Shipments
  (item STRING, count INT)
  WITH (...)
```

```
INSERT INTO Inventory
SELECT item, SUM(count) AS stock
FROM Shipments
GROUP BY item
```

```
CREATE TABLE Inventory
  (item STRING, stock INT)
  WITH (...)
```

item	count
HAT	50
MUG	10
HAT	-15



item	stock
HAT	35
MUG	10

Stream-Table duality

- Some tables experience more than just insert events
- Rows might be updated, or deleted



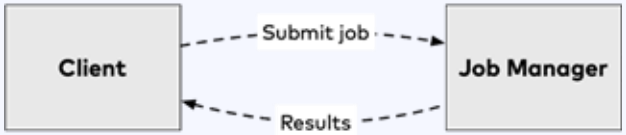
Short name	Long name	Semantics
`+I`	Insertion	Default
`-U`	Update Before	Retract an earlier result
`+U`	Update After	Update an earlier result
`-D`	Delete	Delete an earlier result



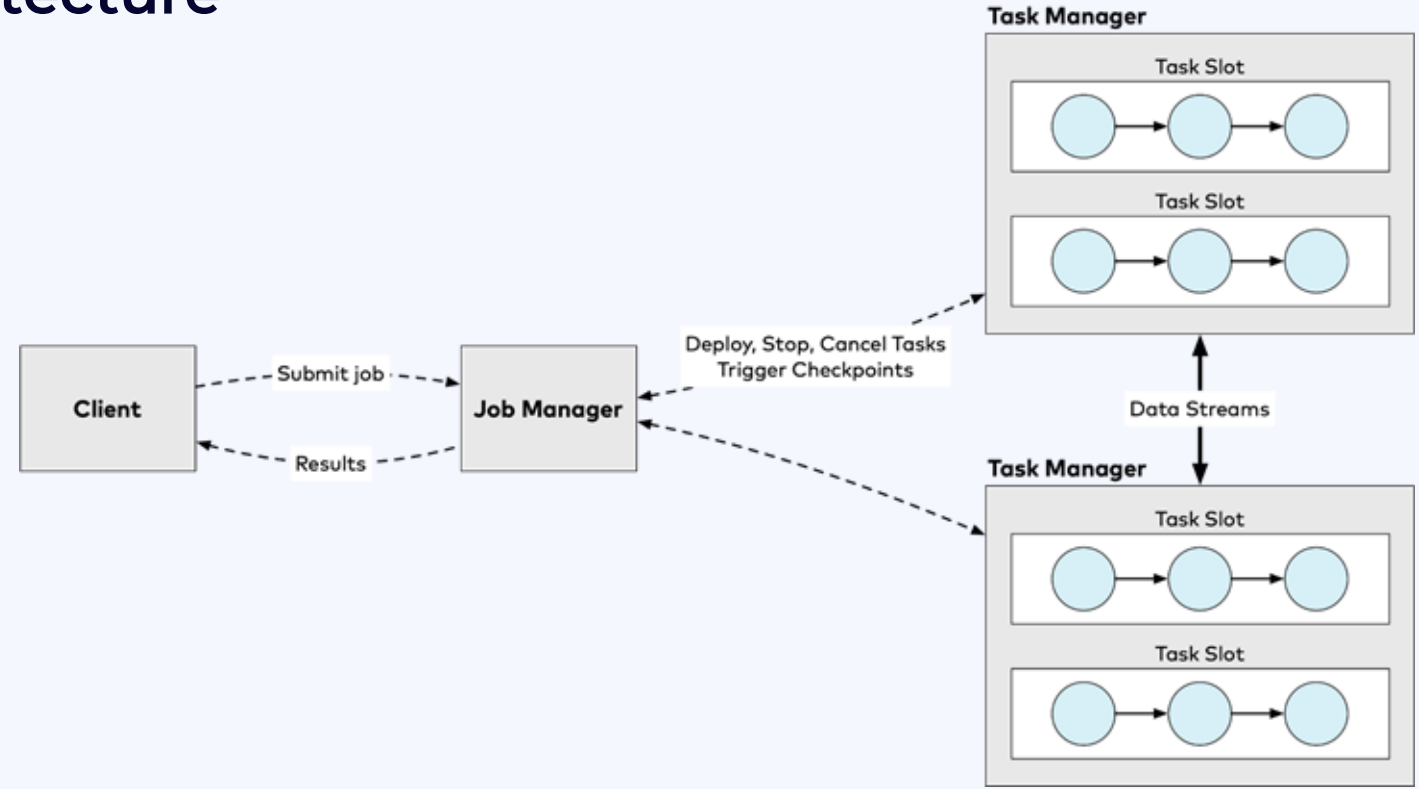
CONFLUENT
Developer

Flink Architecture

Runtime Architecture



Runtime Architecture





CONFLUENT
Developer

Using Kafka with Flink

Tables and topics

```
CREATE TABLE Orders (
    `order_id`    BIGINT,
    `order_time`  TIMESTAMP_LTZ(3) METADATA FROM 'timestamp',
    `price`       DECIMAL(32, 2),
    `quantity`    INT
) WITH (
    'connector' = 'kafka',
    'topic' = 'orders',
    ...
)
```



Flink acts as a compute layer for Kafka, powering real-time applications & pipelines



Flink



Kafka Connect



Kafka



Schema Registry



Kafka Streams



ksqlDB





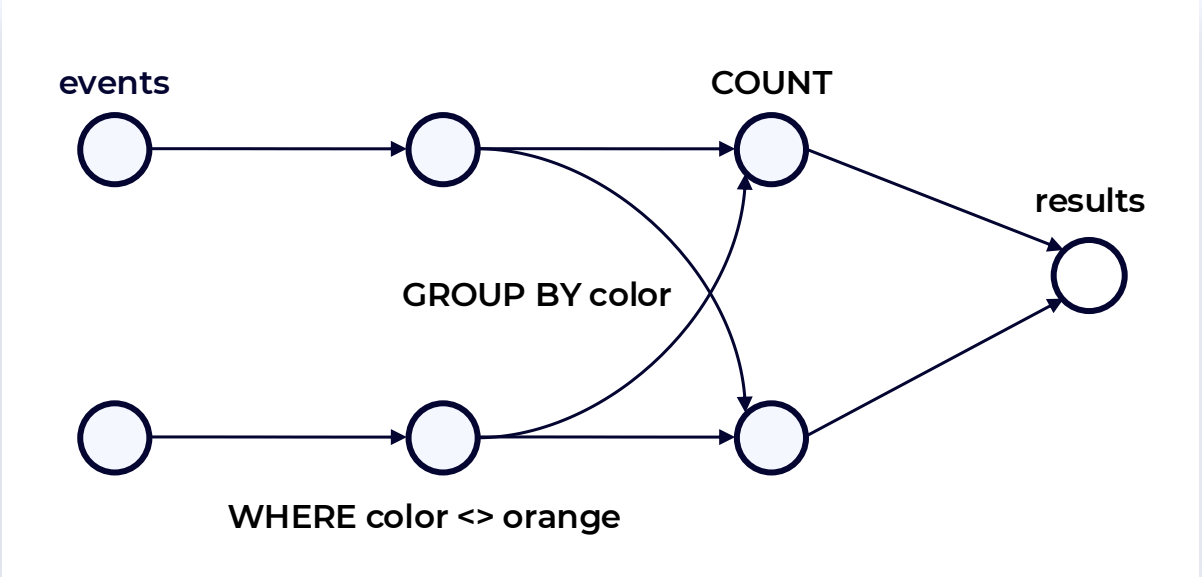
CONFLUENT
Developer

Stateful Stream Processing with Flink SQL

Stateful stream processing with Flink SQL

CONFLUENT Developer

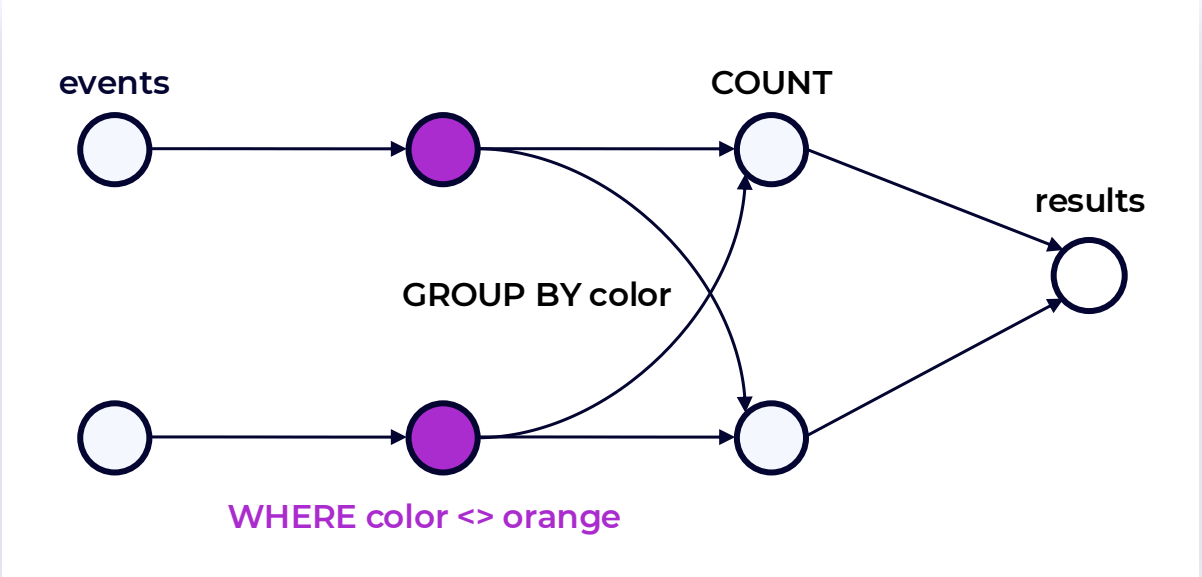
```
INSERT INTO results
SELECT color, COUNT(*)
FROM events
WHERE color <> orange
GROUP BY color;
```



Stateful stream processing with Flink SQL

CONFLUENT Developer

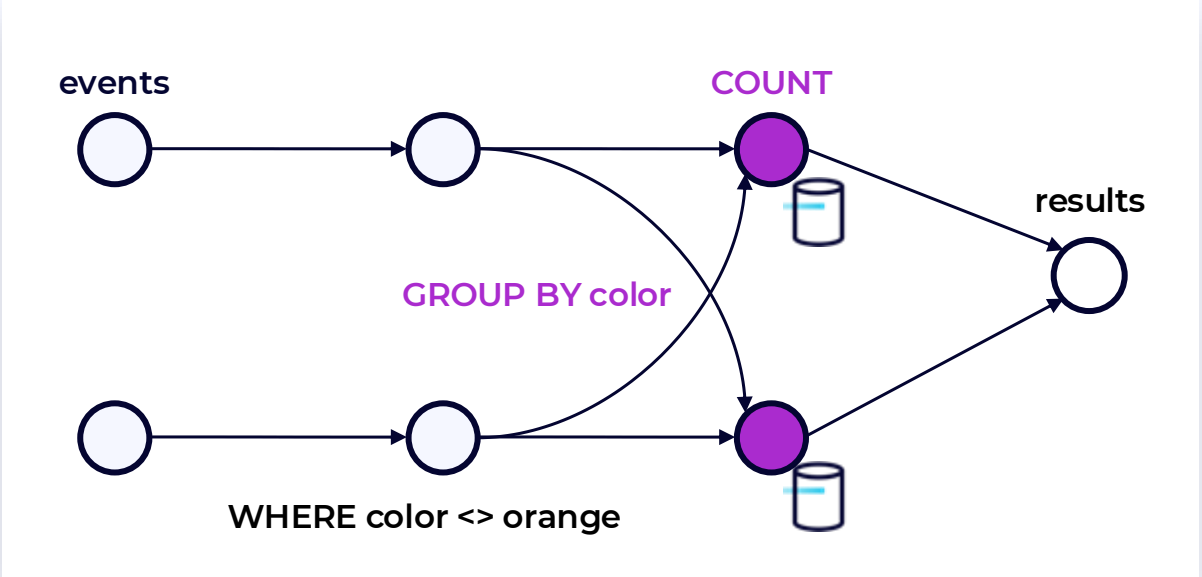
```
INSERT INTO results
SELECT color, COUNT(*)
FROM events
WHERE color <> orange
GROUP BY color;
```



Stateful stream processing with Flink SQL

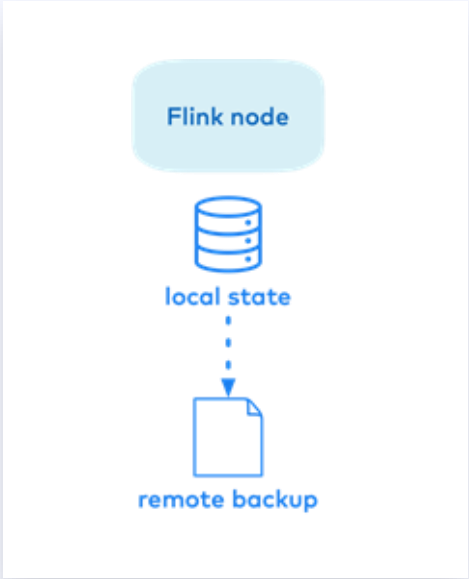
CONFLUENT Developer

- Counting requires state



State

- **Local**
- **Fast**
- **Fault tolerant**



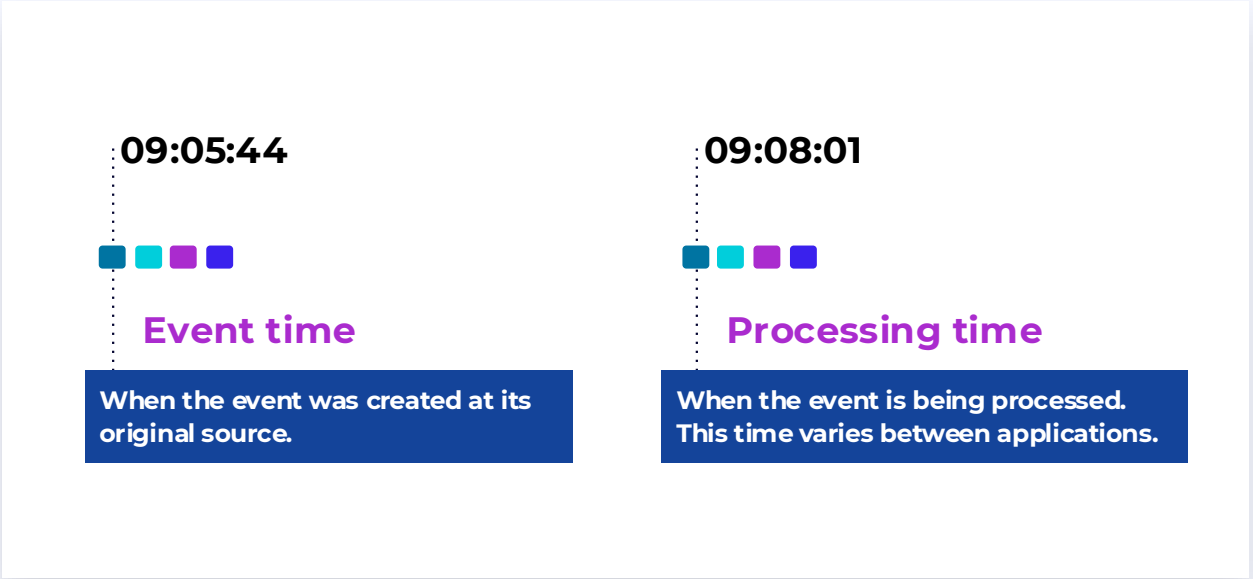


Event Time and Watermarks

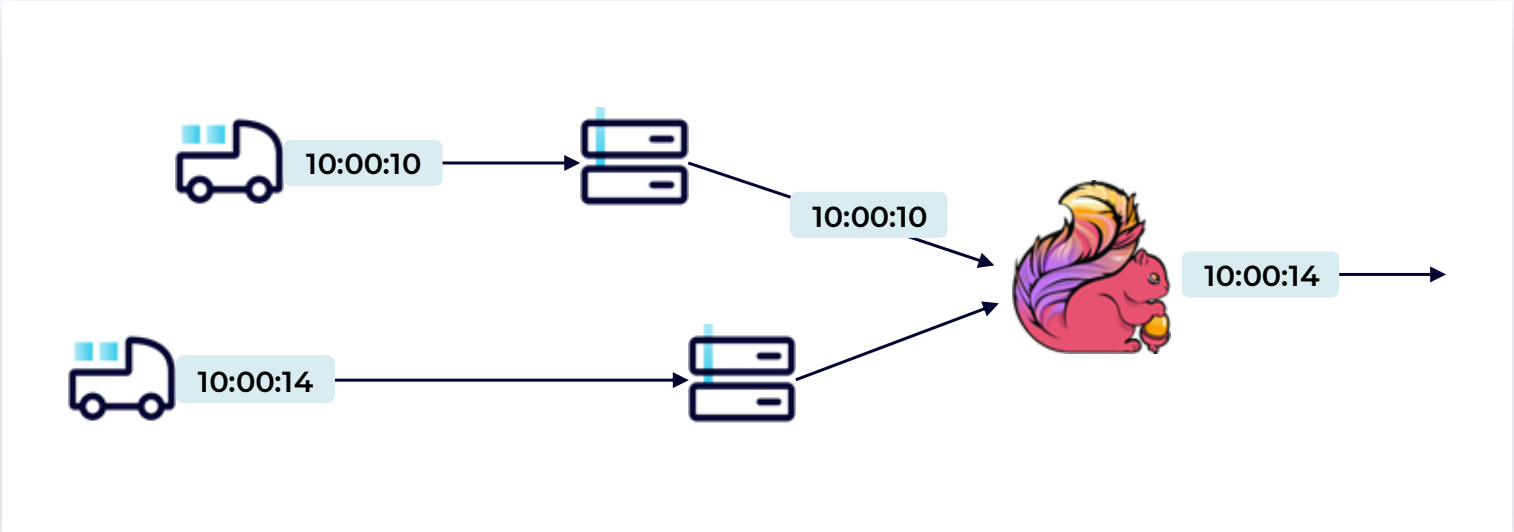
Time

CONFLUENT Developer

- Synchronize
- Wait
- Timeout

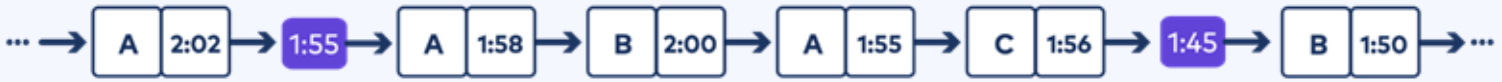


Out-of-order event streams



- Streams are (roughly) ordered by time

Watermarks measure progress of event time



Imagine a window counting events for the hour ending at 2:00. How long should this window wait before producing its results?

It should wait for a ***watermark*** with a timestamp of at least 2:00.



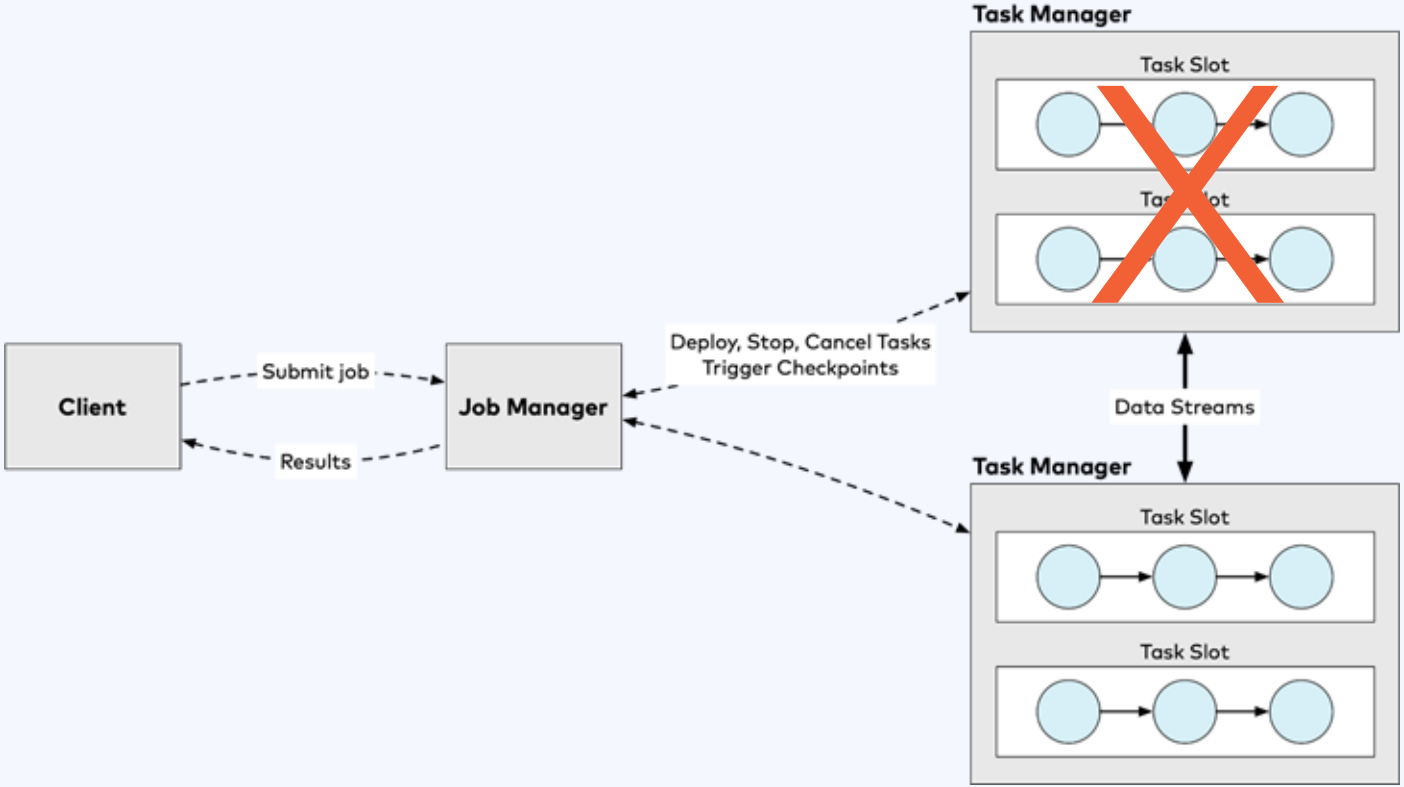
Checkpoints and Recovery

CONFLUENT Developer

A ***checkpoint*** is an automatic snapshot created by Flink, primarily for the purpose of failure recovery

A ***savepoint*** is a manual snapshot created for some operational purpose (e.g., a stateful upgrade)

Failures


Task Manager
Task Slot
Task Slot
Client
Submit job
Results
Job Manager
Deploy, Stop, Cancel Tasks
Trigger Checkpoints
Data Streams
Task Manager
Task Slot
Task Slot

Recovery

Because these are self-consistent, global snapshots

- Flink provides (effectively) exactly-once guarantees
- Recovery involves restarting the entire cluster from the most recent snapshot





Thank you